```
        EndpointFilterDelegate next)                                    ❹
    {
        var id = context.GetArgument<string>(0);                        ❺
        if (string.IsNullOrEmpty(id) || !id.StartsWith('f'))
        {
            return Results.ValidationProblem(
                new Dictionary<string, string[]>
                {
                    {"id", new[]{"Invalid format. Id must start with 'f'"}}
                });
        }

        return await next(context);                                     ❻
    }
}
```

❶ Adds the filter to the endpoint using AddEndpointFilter

❷ The method must return a ValueTask.

❸ context exposes the endpoint method arguments and the HttpContext.

❹ next represents the filter method (or endpoint) that will be called next.

❺ You can retrieve the method arguments from the context.

❻ Calling next executes the remaining filters in the pipeline.

NOTE The `EndpointFilterDelegate` is a named delegate type. It's effectively a `Func<EndpointFilterInvocationContext, ValueTask<object?>>`.

There are many parallels between the middleware pipeline and the filter endpoint pipeline, and we'll explore them in section 5.4.1.

5.4.1 Adding multiple filters to an endpoint

The middleware pipeline is typically the best place for handling cross-cutting concerns such as logging, authentication, and authorization, as these functions apply to all requests. Nevertheless, it can be common to have additional cross-cutting concerns that are endpoint-specific, as we've already